

WHAM-XREX-PFMG₄₇₄

Firmware Build, Flash, and Verification

Standard Operating Procedure

August 2026

Rev	Date	Author	Description
A	2026-08-31	–	Initial issue: build, bootstrap flash, serial reflash. No power-stage procedures yet (Section 2).
B	2026-09-04	–	Added Section 8: uploading a PFM table and firing it – this firmware now drives real HRTIM PWM output (Section 2 updated accordingly). Serial baud raised 9600 → 115200 throughout (Section 7 and elsewhere).

1 Purpose

This document specifies the procedure for building, flashing, and verifying WHAM-XREX-PFMG₄₇₄ firmware, and for confirming a board is running the expected firmware version before it is put into service or handed off.

2 Scope

Covers firmware build and flashing (Sections 4–7) and, as of Rev B, uploading a PWM shot profile and firing it (Section 8) – this firmware **now drives real HRTIM PWM output** on six pins (three complementary phase pairs) the moment FIRE is sent. **This firmware still has no ARM precondition, no state machine, and no fault interlock** – FIRE takes effect immediately and unconditionally whenever it is sent, by design (see docs/command_reference.md). Treat any gate driver or power stage wired to these pins as live for the duration of Section 8, per your lab’s normal HV/gate-driver procedure, exactly as if driven by any other PWM source – this SOP provides no closed-loop protection behind it. This document still does not cover power-stage bring-up, calibration, or safety-interlock

procedures beyond that caution; a companion procedure remains needed once a state machine/interlock exists in firmware.

3 Prerequisites

3.1 Equipment

Item	Notes
WHAM-XREX-PFMG474 board	Powered per its own hardware documentation (not covered here).
ST-Link (or equivalent SWD debug probe)	Required for the bootstrap flash (Section 5) and for recovery if a board stops responding over serial entirely.
USB-to-serial adapter	Wired to USART2: adapter TX → PA3 (USART2_RX), adapter RX → PA2 (USART2_TX), GND ↔ GND.

(TX/RX wiring above is crossed: adapter TX goes to the MCU's RX pin and vice versa, as usual for a point-to-point serial link.)

3.2 Software

Tool	Notes
STM32CubeIDE stm32flash	Project build and the bootstrap SWD flash. AN3155 USART bootloader client. macOS: brew install stm32flash.
Python 3 + pyserial	Runs python/wham_serial_flash.py. pip install pyserial.

CAUTION: This firmware has not yet been through an ESD-controlled production process. Use standard ESD precautions (grounding strap, static-safe surface) when handling the bare board.
--

4 Procedure: Build the Firmware

1. Open the project in STM32CubeIDE and press **Refresh** (F5) on the project if any files were added outside the IDE since it was last opened.
2. Build the **Debug** configuration (Project → Build Project, or from the command line: `cd Debug && make all -j4`).
3. Confirm the build produced no warnings. A clean build produces WHAM-XREX-PFMG474.elf in Debug/.

4. **Generate the .bin.** This project's build configuration does not have the "Convert to binary file" post-build step enabled, so it must be generated by hand:

```
arm-none-eabi-objcopy -O binary \  
  Debug/WHAM-XREX-PFMG474.elf \  
  Debug/WHAM-XREX-PFMG474.bin
```

5 Procedure: Bootstrap Flash (SWD)

Required the *first* time a board receives firmware with the serial-bootloader feature (BOOT command), since that feature is what the serial reflash path in Section 6 depends on – a board cannot use it to receive the very first firmware that adds it. Also use this procedure to recover a board that no longer responds to BOOT over serial at all.

1. Connect the ST-Link to the board's SWD header and to the host computer.
2. In STM32CubeIDE, press **Run** (or **Debug**) to build and flash via the ST-Link, as normal for this IDE.
3. Proceed to Section 7 to confirm the board is running the expected firmware.

6 Procedure: Serial Reflash

Use this for all firmware updates *after* the bootstrap flash in Section 5 has been done once. This is the remote-friendly path: no physical BOOTo/NRST/SWD access is required, only the USART2 serial link.

1. With the board powered and running its current firmware, and the USB-serial adapter connected, run:

```
python3 python/wham_serial_flash.py \  
  --port /dev/cu.usbserial-XXXXX
```

substituting the actual serial device path. (On macOS, a single USB-serial adapter shows up as both a `cu.*` and `tty.*` device node; the script sees this as two candidates and will not auto-detect – `--port` must be passed explicitly. Prefer the `cu.*` node.)

2. The script will: send BOOT and wait for the acknowledgement; run `stm32flash` to write and verify `Debug/WHAM-XREX-PFMG474.bin`; then start the new firmware automatically (no manual reset needed – see Section 6.1).

3. Confirm the script reports [done] with firmware written and verified before proceeding.
4. Proceed to Section 7.

6.1 Note on automatic restart after flashing

Early testing of this procedure found that the newly-flashed firmware could come up unresponsive over serial until the board was manually power-cycled, because the STM32 ROM bootloader's *Go* command (used to start the new firmware without a physical reset) does not perform a true chip reset, and left the processor's vector table pointer aimed at the bootloader's own vector table rather than the application's. This has been fixed in firmware (`boot_jump.c`) – a manual reset should not be necessary. If a board flashed under this SOP does not respond to the verification step below without a manual reset, that is a regression and should be reported, not treated as expected behavior.

7 Procedure: Verification

1. Open a serial connection to the board at **115200 8N1** (raised from 9600 as of Rev B – see `docs/changelog.txt`).
2. Send `*IDN?` followed by a line ending (`\r`, `\n`, or both).
3. Confirm a reply of the form:

`OK <board name> <hardware rev> <firmware version>`

e.g. `OK WHAM-XREX-PFMG474 REVA v0.1.`

4. Confirm the reported firmware version matches the version that was just built/flushed.

Acceptance criterion for this SOP: Step 3 above returns a well-formed reply with the expected version, on the first query, with no manual reset performed since the flash in Section 4 or 6.

8 Procedure: Upload and Fire a PFM Table

New in Rev B. Uploads a complete PWM shot profile (a sequence of period/duty entries, one per switching cycle) and plays it back on demand. Full protocol detail: `docs/command_reference.md`'s `TABLE:*` and `FIRE` entries.

CAUTION: FIRE drives real HRTIM PWM output on six pins (three complementary phase pairs, ~ 100 ns hardware dead time between each pair) the instant it is sent. There is no ARM step, no state machine, and no fault interlock in this firmware – FIRE always takes effect immediately, whether the board was idle or a previous shot was still running. Treat any gate driver or power stage wired to these pins as live before proceeding, per your lab’s normal HV/gate-driver procedure.

1. Complete Sections 4–7 first – a board must already be running current, verified firmware.
2. Table *construction* (what shot profile to build) lives entirely on the host, not in firmware – see pfm.h’s own header note for why. `python/pfm_table_upload.py` is the reference builder: edit its `PROFILE` constant (1–5; see the script’s own header comment for what each one is) before running:

```
python3 python/pfm_table_upload.py \  
--port /dev/cu.usbserial-XXXXX
```
3. Confirm the script reports [done] table uploaded and confirmed with the expected entry count (TABLE:END -> OK <count>).
4. Optionally confirm independently at any time with TABLE? over a serial terminal (e.g. `python/scpi.py`) – replies OK <count>, matching the upload.
5. With the CAUTION above acknowledged and anything downstream of these six pins confirmed safe to drive, send FIRE. A prompt OK confirms the shot started; output stops on its own, at a coherent period boundary, once the table is exhausted – there is no separate STOP command and no completion notification.
6. Sending FIRE against an empty table (nothing uploaded yet) is rejected with ERR 5 and drives nothing – this is the one built-in safeguard, not a substitute for the interlock discussed above.

Acceptance criterion for this SOP: TABLE:END reports the exact entry count the host script intended to send, and FIRE returns OK with no ERR. Confirming the actual output waveform (dead time, phase spacing, duty accuracy) is a bench/instrumentation task outside this SOP’s scope – see `docs/test_protocol.md`’s T4 group if that level of verification is needed.

9 Troubleshooting

Symptom	Cause / action
stm32flash reports Failed to init device, timeout. before any write occurs	Not a firmware issue – this probe doesn't depend on what's running on the chip. Check board power, serial wiring (TX/RX not crossed, common GND), and that no other program has the port open.
BOOT gets no reply, but power/wiring check out	The firmware currently on the board may predate the BOOT command (run the bootstrap flash, Section 5) or may be a board affected by the Go-jump issue in Section 6.1 – power-cycle once, then retry.
Verification (Section 7) fails after a serial reflash	Power-cycle the board once and repeat verification. If it then succeeds, this is the regression described in Section 6.1 – report it. If it still fails, treat as a failed flash and repeat Section 6 from the start.

10 References

- AGENTS.md – project orientation and current functionality.
- docs/command_reference.md – full serial command protocol, including TABLE:* and FIRE (Section 8).
- docs/serial_reflash_guide.md – detailed reflash mechanism, including the Go-jump bug referenced in Section 6.1.
- docs/test_protocol.md – bench-level verification, including waveform checks beyond this SOP's acceptance criteria.
- python/pfm_table_upload.py – reference table builder used in Section 8; see its own header comment for what each PROFILE value builds.
- docs/changelog.txt – dated development history.